PROJECT ANALYSIS AND DESIGN · OVERVIEW

Thiqah: service marketplace

A project analysis and design report based on the implemented Flutter application.

TRUST

Connecting customers with service providers
to complete paperwork and administrative tasks.

Project scope

Customers publish requests, compare offers, select a provider, simulate payment, confirm completion and leave a review. Providers discover published requests, submit offers and deliver work. Administrators manage local user accounts.

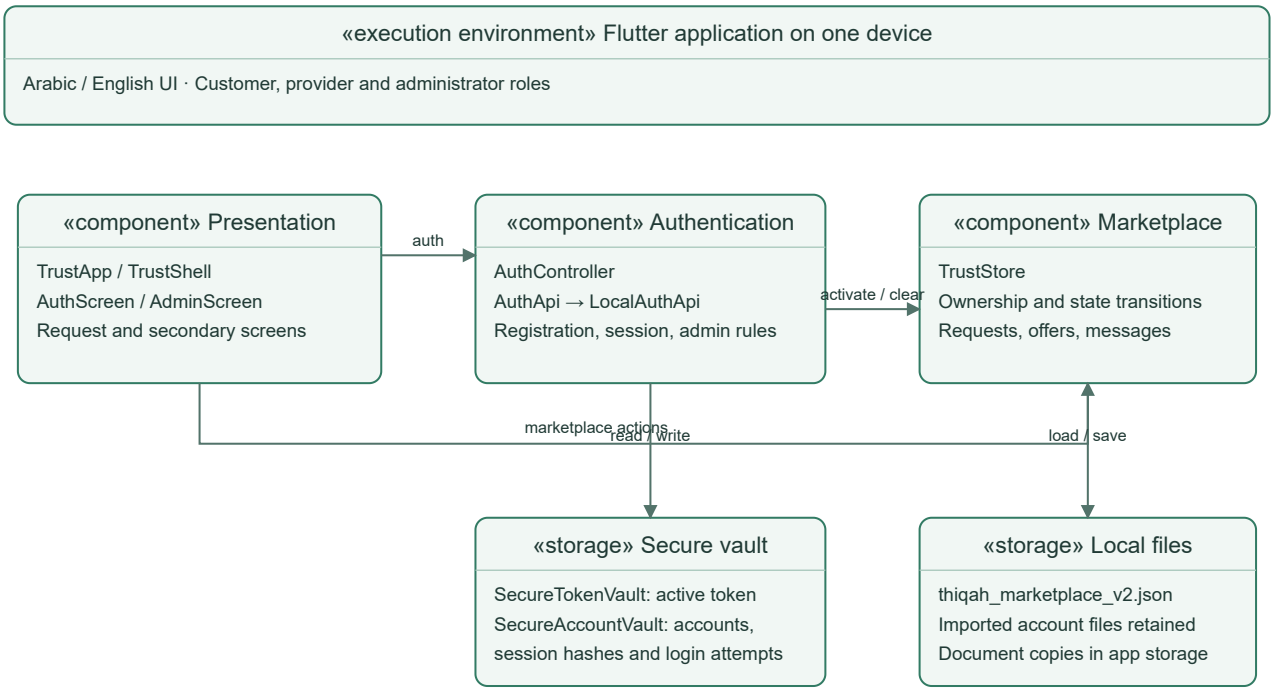
Required deliverable	Report section
UML model	1 · Component / architecture model
Use case diagram	2 · Actors and supported operations
Activity diagram	3 · Request fulfillment workflow
Sequence diagram	4 · Offer submission and persistence
Database diagram (if any)	5 · Actual storage and proposed relational schema
Class diagram	6 · Main Dart classes and relationships
ER diagram	7 · Conceptual entities and cardinalities

> **Implementation boundary:** Accounts and marketplace data exist on one device. There is no cross-device synchronization, real payment processing or OTP verification. The separate Next.js project in `Trust/` is not the implementation modeled in this report.

PROJECT ANALYSIS AND DESIGN · 1

UML model

UML component view: the application is separated into presentation, authentication, domain logic and device storage.



> UML is a modeling language, not one specific diagram. This section uses a component view; the following sections provide behavioral and structural UML views.

Key requirements

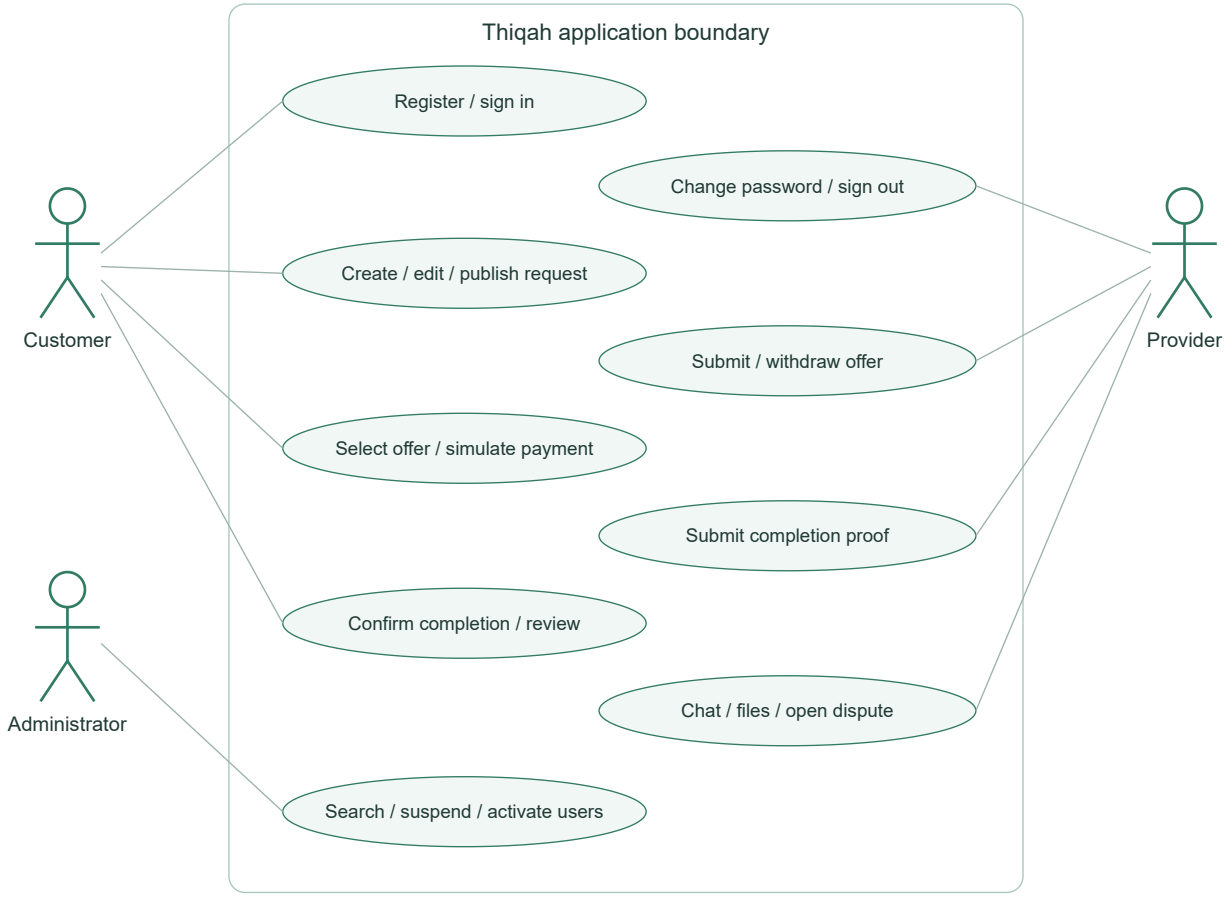
Functional: registration and login; request publishing and drafts; offers; simulated payment; fulfillment; chat and documents; reviews; account suspension.

Quality constraints: Arabic/English support, role and ownership checks, protected account storage, serialized file writes, and preservation of unreadable original data.

PROJECT ANALYSIS AND DESIGN · 2

Use case diagram

Primary actor responsibilities. Shared actions and access conditions are detailed below.



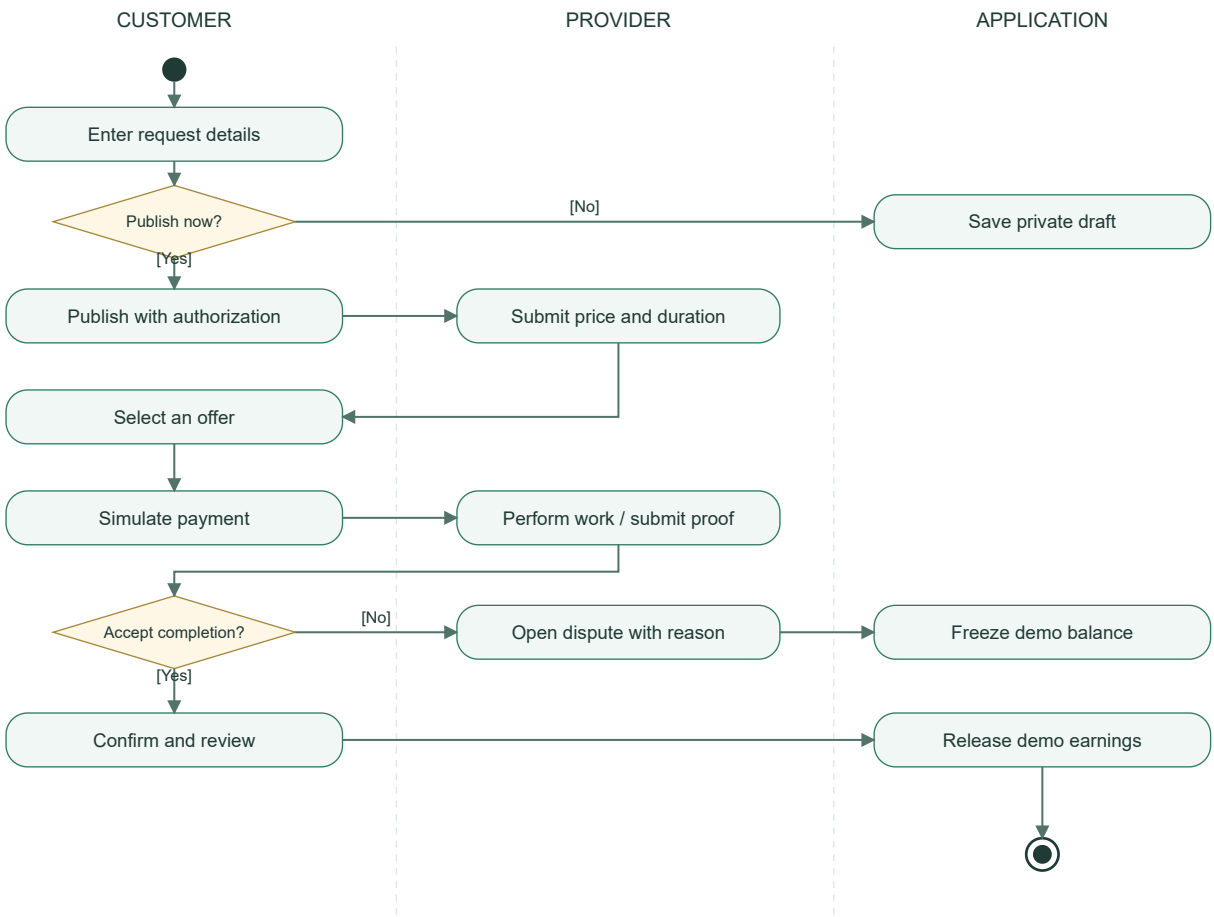
Actor	Additional shared actions and constraints
Customer	Sign out, change password, chat, attach/export files and open a dispute for own selected request.
Provider	Register/sign in, view the market, chat/files/dispute for selected work; simulate withdrawal of earned funds.
Administrator	Sign in, change password, search accounts and change user status; cannot suspend an administrator.

Associations show the principal use cases, not the chronological order. Chat requires a selected offer. Private request details are visible to the owner and the selected provider.

PROJECT ANALYSIS AND DESIGN · 3

Activity diagram

Main request workflow with customer, provider and application responsibilities.



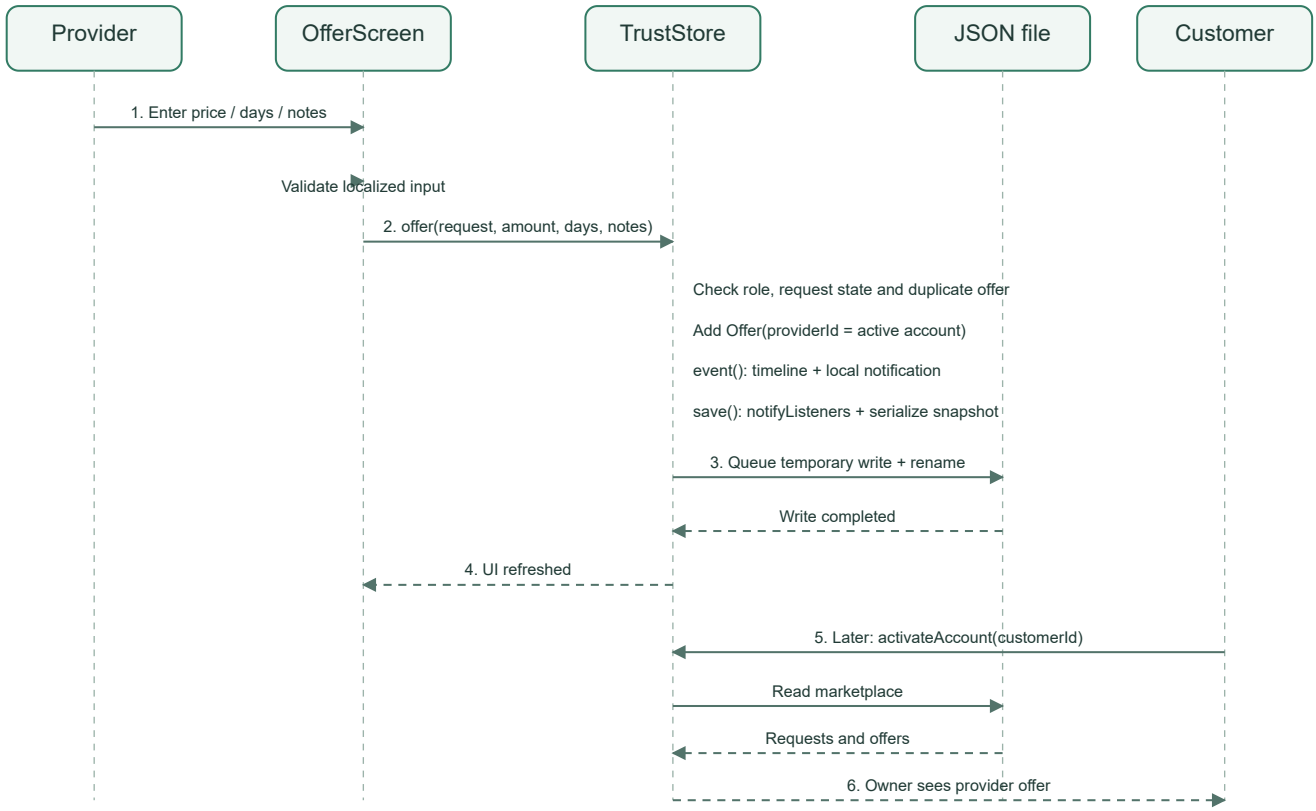
Precondition: valid signed-in account; role switches occur on the same device. Invalid forms remain on the form with validation feedback. A draft can later be published. Either selected participant can open a dispute while active or awaiting confirmation. Dispute settlement is not implemented; this branch remains pending.

Other route: the customer may cancel a request in draft, open or pending-payment state. Successful completion is the only branch shown ending at the activity final node.

PROJECT ANALYSIS AND DESIGN · 4

Sequence diagram

Scenario: a provider submits an offer, and the customer sees it after switching accounts.



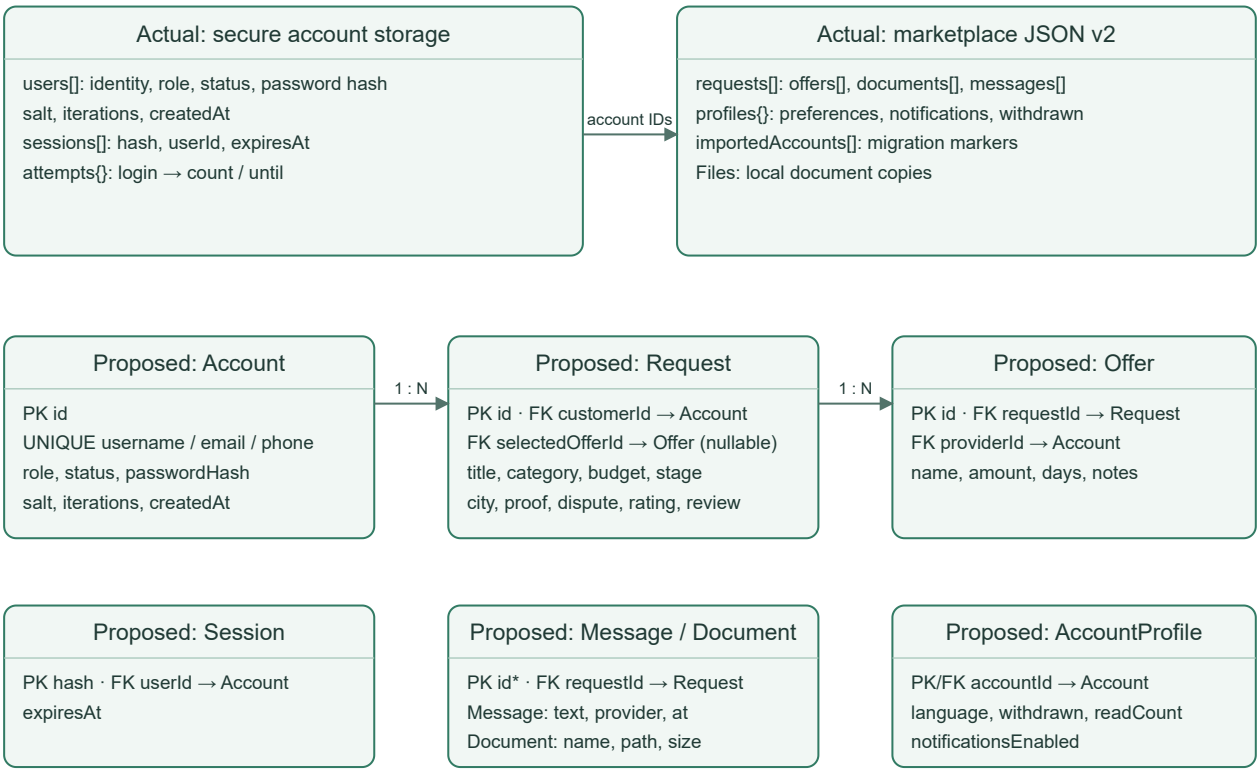
Persistence timing: `offer()` is synchronous and calls `save()` indirectly. Disk writes are queued; the UI does not wait for disk completion. Account switching waits for queued writes. The numbered interaction is a simplified successful sequence, not a claim of synchronous disk confirmation.

Condition	Alternative behavior
Invalid amount or duration	The form reports validation errors; no offer is submitted.
Wrong role, closed request, duplicate offer	TrustStore rejects the operation with StateError.
Local write fails	The store sets storageError and notifies the UI.
Non-owner customer	The request is filtered out of that customer’s visible requests.

PROJECT ANALYSIS AND DESIGN · 5

Database diagram

The Flutter implementation uses device storage, not a relational database server.



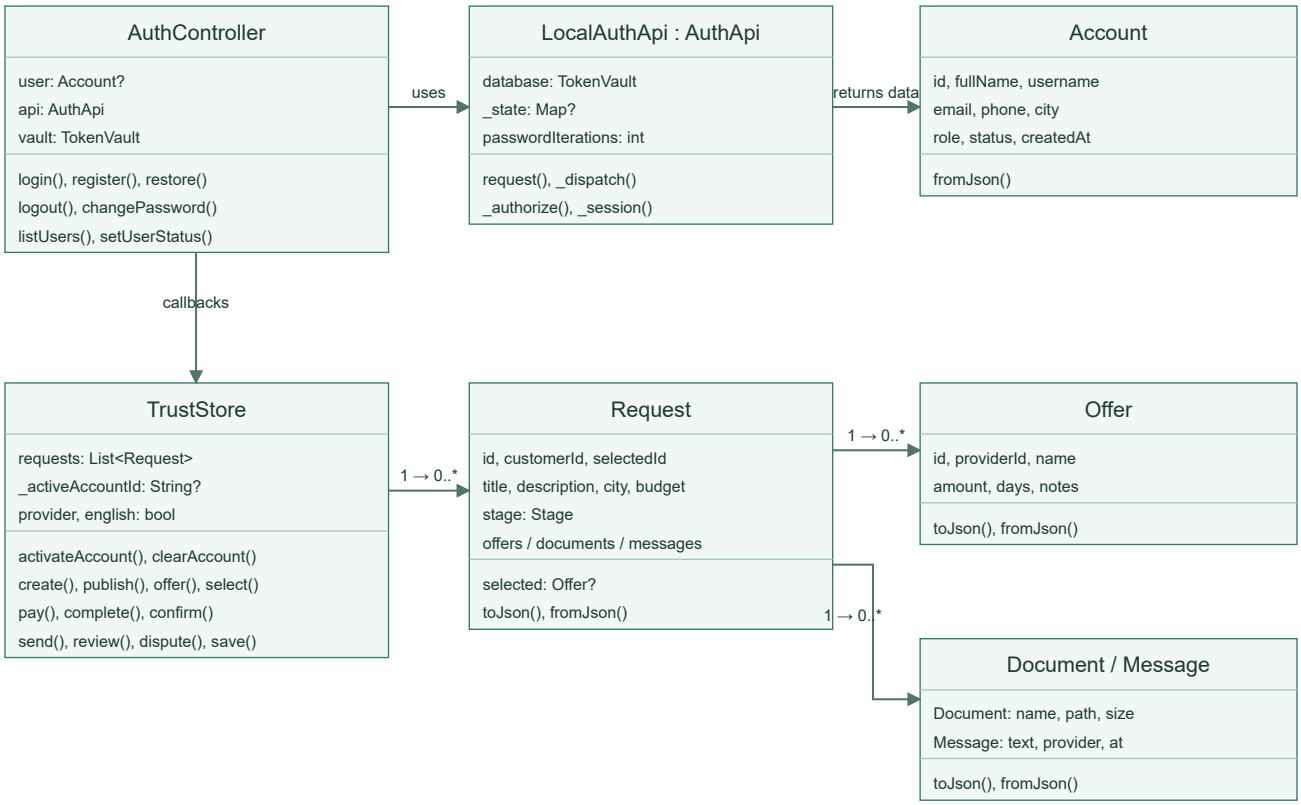
> **Proposed tables are a design translation, not existing SQL tables.** An explicit Request foreign key is added to nested records; generated IDs (*) are required for Message and Document. In the current JSON format they have no IDs. Categories are fixed lists, and review/dispute data are fields on Request.

Relational constraints: selectedOfferId must refer to an offer for the same request; offer provider must have the provider role; customerId must refer to a customer. Enforce these domain rules in application logic and transactions.

PROJECT ANALYSIS AND DESIGN · 6

Class diagram

Selected implemented Dart classes; members are abbreviated for readability.



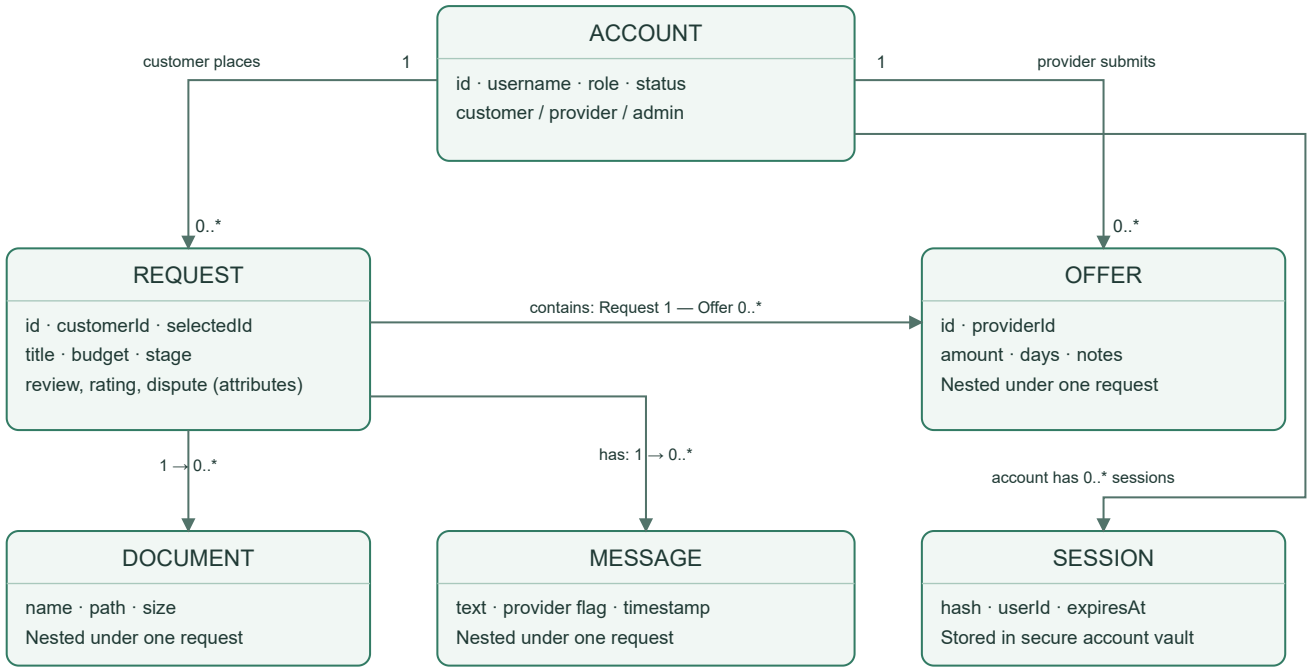
Arrows show directed associations/dependencies, with multiplicities for collections. Request owns nested offers, documents and messages in serialized storage. LocalAuthApi extends AuthApi; SecureTokenVault and SecureAccountVault implement TokenVault. AuthController and TrustStore extend ChangeNotifier.

State enumeration: draft, open, payment, active, confirmation, completed, disputed, refunded, cancelled. `refunded` exists in the enum but has no refund transition implemented in TrustStore.

PROJECT ANALYSIS AND DESIGN · 7

ER diagram

Conceptual relationships derived from the local account and marketplace data model.



Relationship	Cardinality and rule
Customer → Request	One customer owns zero or many requests; each signed-in request has one customer.
Provider → Offer	One provider submits zero or many offers; each offer has one provider.
Request → Offer	One request has zero or many offers and selects zero or one of those offers.
Request → Message / Document	Each request contains zero or many messages and documents; no independent IDs in current storage.
Account → Session	One account has zero or many sessions; each session belongs to one account.

Cardinalities describe the authenticated marketplace. Legacy anonymous demo records are outside this model. A request can store at most one rating/review and one dispute reason. The message sender is stored as a provider/customer flag, not a user foreign key.

PROJECT ANALYSIS AND DESIGN · REFERENCE

Source mapping and validation

How the diagrams map to the application and its documented behavior.

Source file	Evidence used
trust_app/README.md	Current project scope, workflow, storage format, limitations and documented checks.
trust_app/lib/main.dart	App bootstrap, identity callbacks, routing and localization.
trust_app/lib/src/auth.dart	Account model, authentication controller and token-vault interface.
trust_app/lib/src/local_auth.dart	Local account records, hashing, sessions, registration and administrator permissions.
trust_app/lib/src/store.dart	Models, ownership, request states, shared marketplace, serialization and migration.
trust_app/lib/src/request_screens.dart	Request forms, localized number input and attachment copying.
trust_app/lib/src/secondary_screens.dart	Messaging, wallet and document views/export.
trust_app/test/marketplace_test.dart	Cross-account request visibility, selected-provider permissions and migration scenarios.
trust_app/test/marketplace_widget_test.dart	Provider offer submission and later visibility to the customer.

Design assumptions

- The report models the current Flutter university project, not the commercial roadmap or separate web application.
- The database section distinguishes actual JSON/secure storage from a proposed relational conversion.
- The diagrams summarize selected public workflows; they are not a claim that all planned production services are implemented.
- The component model supplies the general “UML MODEL” requirement; use case, activity, sequence and class views provide its detailed models.

Verification status

This report was prepared from source inspection. No application tests were rerun for this documentation task. The README records 33 previously passing tests; that is historical project documentation, not a new test result.